

 **jihoonkim888 / PyTopo3D** Public[<> Code](#) [Issues 2](#) [Pull requests](#) [Actions](#) [Projects](#) [Security and quality](#) [Insights](#)

About

PyTopo3D: 3D SIMP Topology Optimization Framework for Python

[Readme](#)[MIT license](#)[Activity](#)[70 stars](#)[2 watching](#)[13 forks](#)[Report repository](#)

Releases

No releases published

Packages

No packages published

Contributors 2

-  **jihoonkim888** Jihoon Kim
-  **narnia-ai-daniel** Daniel Kim

Languages

● Python 99.1% ● Shell 0.9%

 main ▾

 2 Branches

 0 Tags





	narnia-ai-daniel Refactor GPU solver in solver.py to remove unnecessary parameters  02e4fdf · last year 
 assets	Add files via upload last year
 build-tools	Add dependencies and explanation for ... last year
 examples	Add files via upload last year
 pytopo3d	Refactor GPU solver in solver.py to re... last year
 scripts	Add animation feature of the optimizati... last year
 .gitignore	Reorganize package files into build-tool... last year
 LICENSE	Create LICENSE last year
 README.md	Add dependencies and explanation for ... last year
	Update environment.yml for GPU supp... last year

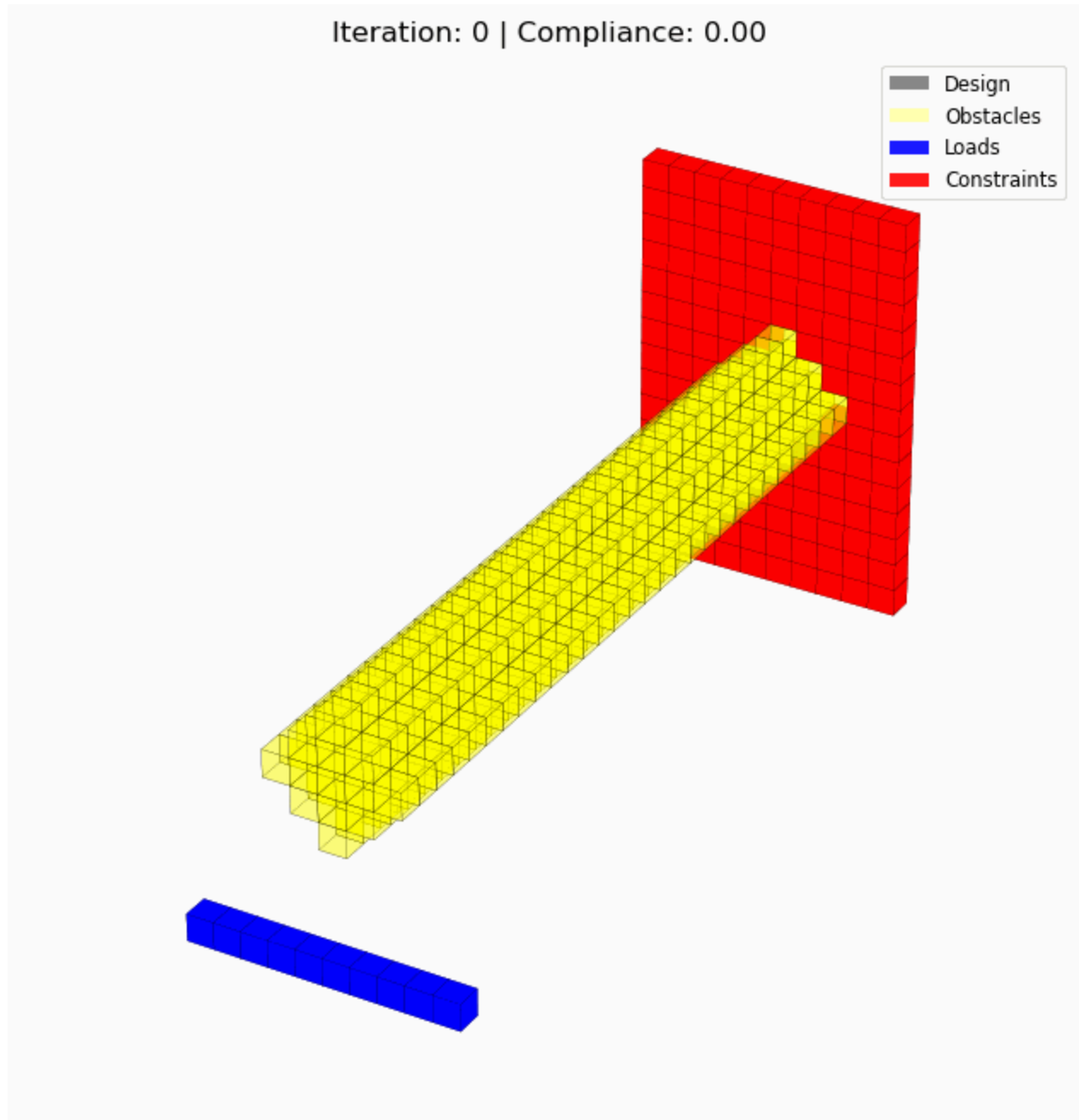
 environment.yml main.py

Add CuPy for GPU acceleration option

last year

 **README** MIT license

PyTopo3D: 3D SIMP Topology Optimization Framework for Python



A comprehensive Python implementation of 3D Topology Optimization based on SIMP (Solid Isotropic Material with Penalization) method. Unlike traditional MATLAB implementations, PyTopo3D brings the power of 3D SIMP-based optimization to the Python ecosystem with support for obstacle regions.

Table of Contents

- [PyTopo3D: 3D SIMP Topology Optimization Framework for Python](#)
 - [Table of Contents](#)
 - [Overview](#)
 - [Installation](#)
 - [Basic Usage](#)
 - [Configuration Parameters](#)
 - [Command-line Interface](#)
 - [Python Package API](#)
 - [Advanced Features](#)
 - [Obstacle Configuration](#)
 - [Command Line Usage](#)
 - [Python API Usage](#)
 - [Obstacle Configuration Format](#)
 - [Design Space Customization](#)
 - [Command Line Usage](#)
 - [Python API Usage](#)
 - [Understanding Pitch and Resolution](#)
 - [Exporting Results](#)
 - [Command Line Usage](#)
 - [Animation Generation](#)
 - [GPU Acceleration](#)
 - [Installation with GPU Support](#)

- [Enabling GPU Acceleration](#)
- [Experiment Management](#)
 - [Command Line Options](#)
- [Acknowledgements](#)
- [Citation](#)
- [Roadmap](#)
 - [Version 0.2.0 \(Performance & Interface\)](#)
 - [Version 0.3.0 \(Core Functionality Improvements\)](#)
 - [Version 0.4.0 \(Pre-release Stabilization\)](#)
 - [Version 1.0.0 \(Stable Release\)](#)

Overview

This code performs 3D structural topology optimization using the SIMP (Solid Isotropic Material with Penalization) method. It is designed to be efficient by utilizing:

- Parallel solver (PyPardiso if available, otherwise SciPy's spsolve)
- Precomputed assembly mapping for fast matrix assembly
- Support for obstacle regions where no material can be placed
- Flexible obstacle configuration via JSON files
- Advanced visualization capabilities including animation generation

Installation

You can install PyTopo3D in two ways:

Option 1: Install via pip

```
# Basic installation
pip install pytopo3d

# With GPU acceleration support
pip install pytopo3d[gpu]
```



Option 2: Clone the repository

1. Clone this repository:

```
git clone https://github.com/jihoonkim888/PyTopo3D.git
cd PyTopo3D
```



2. Create and activate the conda environment:

```
# Create the environment from the environment.yml file
conda env create -f environment.yml

# Activate the environment
conda activate pytopo3d
```



3. For developers, install in development mode:

```
# Basic installation
cd build-tools

pip install -e .

# With GPU acceleration support
pip install -e ".[gpu]"
```



Basic Usage

Configuration Parameters

The main optimization parameters are:

- `nelx` , `nely` , `nelz` : Number of elements in x, y, z directions (default: 60, 30, 20)
- `volfrac` : Volume fraction constraint (0.0-1.0) (default: 0.3)
- `penal` : Penalization power for SIMP method (default: 3.0)
- `rmin` : Filter radius for sensitivity filtering (default: 3.0)
- `disp_thres` : Display threshold for 3D visualization (elements with density > `disp_thres` are shown) (default: 0.5)
- `tolx` : Convergence tolerance on design change (default: 0.01)
- `maxloop` : Maximum number of iterations (default: 2000)

Command-line Interface

To run a basic optimization:

```
python main.py --nelx 32 --nely 16 --nelz 16 --volfrac 0.3 --penal 3.0 --rmin 3.0
```



For full options:

```
python main.py --help
```



Python Package API

```
import numpy as np
from pytopo3d.core.optimizer import top3d
```




```
# Define parameters
nelx, nely, nelz = 32, 16, 16
volfrac = 0.3
penal = 3.0
rmin = 3.0
disp_thres = 0.5

# Run optimization
result = top3d(nelx, nely, nelz, volfrac, penal, rmin, disp_thres)

# Save result
np.save("optimized_design.npy", result)
```

Advanced Features

Obstacle Configuration

PyTopo3D supports defining regions where material should not be placed during optimization.

Command Line Usage

```
python main.py --obstacle-config examples/obstacles_config_cylinder.json
```



Python API Usage

```
# Create a custom obstacle mask
obstacle_mask = np.zeros((nely, nelx, nelz), dtype=bool)
obstacle_mask[5:15, 3:7, 3:7] = True # Example obstacle

# Or load obstacles from config file
from pytopo3d.preprocessing.geometry import load_geometry_data
```



```
design_space_mask, obstacle_mask, combined_obstacle_mask = load_geometry_data(
    nelx=nelx,
    nely=nely,
    nelz=nelz,
    obstacle_config="path/to/config.json"
)

# Use the mask in optimization
result = top3d(nelx, nely, nelz, volfrac, penal, rmin, disp_thres,
               obstacle_mask=combined_obstacle_mask)
```

Obstacle Configuration Format

The obstacle configuration file is a JSON file with the following structure:

```
{
  "obstacles": [
    {
      "type": "cube",
      "center": [0.5, 0.5, 0.2], // x, y, z as fractions [0-1]
      "size": 0.15              // single value for a cube
    },
    {
      "type": "sphere",
      "center": [0.25, 0.25, 0.6],
      "radius": 0.1
    },
    {
      "type": "cylinder",
      "center": [0.75, 0.5, 0.5],
      "radius": 0.08,
      "height": 0.7,
      "axis": 2                // 0=x, 1=y, 2=z
    },
  ],
}
```



```
{
  "type": "cube",
  "center": [0.25, 0.75, 0.5],
  "size": [0.15, 0.05, 0.3] // [x, y, z] for a cuboid
}
```

Supported obstacle types:

- `cube` : A cube or cuboid. Use `size` as a single value for a cube, or as `[x, y, z]` for a cuboid.
- `sphere` : A sphere. Use `radius` to set the size.
- `cylinder` : A cylinder. Use `radius`, `height`, and `axis` (0=x, 1=y, 2=z) to configure.

All positions are specified as fractions [0-1] of the domain size, making it easy to reuse configurations across different mesh resolutions.

Design Space Customization

PyTopo3D allows using STL files to define the design space geometry, enabling complex shapes beyond the standard rectangular domain.

Command Line Usage

```
python main.py --design-space-stl path/to/design_space.stl --pitch 0.5
```



Command line options:

- `--design-space-stl` : Path to an STL file defining the design space geometry
- `--pitch` : Distance between voxel centers when voxelizing STL (default: 1.0, smaller values create finer detail)
- `--invert-design-space` : Flag to invert the design space (treat STL as void space rather than design space)

Python API Usage

```
from pytopo3d.preprocessing.geometry import load_geometry_data
import numpy as np

# Load design space from STL
design_space_mask, obstacle_mask, combined_obstacle_mask = load_geometry_data(
    nelx=60,
    nely=30,
    nelz=20,
    design_space_stl="path/to/design_space.stl",
    pitch=0.5,
    invert_design_space=False
)

# The shape of the mask is determined by the STL geometry and pitch
nely, nelx, nelz = design_space_mask.shape
print(f"Resolution from voxelization: {nely}x{nelx}x{nelz}")

# Use the mask in optimization
from pytopo3d.core.optimizer import top3d

result = top3d(
    nelx=nelx,
    nely=nely,
    nelz=nelz,
    volfrac=0.3,
    penal=3.0,
    rmin=3.0,
    disp_thres=0.5,
    obstacle_mask=combined_obstacle_mask
)
```



Understanding Pitch and Resolution

The `pitch` parameter directly controls the resolution of the voxelized model:

- Smaller pitch values create higher resolution voxelizations
- The number of voxels along any dimension = physical length ÷ pitch
- Choose pitch value based on the level of detail needed and computational resources available

Exporting Results

You can export the final optimization result as an STL file for 3D printing or further analysis in CAD software.

Command Line Usage

```
python main.py --nelx 32 --nely 16 --nelz 16 \  
    --volfrac 0.3 --penal 3.0 --rmin 3.0 \  
    --export-stl \  
    [--stl-level 0.5] \  
    [--smooth-stl] \  
    [--smooth-iterations 5]
```



Options:

- `--export-stl` : Flag to enable STL export of the final optimization result
- `--stl-level` : Contour level for the marching cubes algorithm (default: 0.5)
- `--smooth-stl` : Flag to apply Laplacian smoothing to the mesh (default: True)
- `--smooth-iterations` : Number of iterations for mesh smoothing (default: 5)

Animation Generation

PyTopo3D can generate animations of the optimization process.

```
python main.py --nelx 32 --nely 16 --nelz 16 \  
    --create-animation \  
    --animation-frequency 10 \  
    --animation-frames 50 \  
    --animation-fps 5
```



Options:

- `--create-animation` : Flag to enable animation generation
- `--animation-frequency` : Store every N iterations for the animation (default: 10)
- `--animation-frames` : Target number of frames in the final animation (default: 50)
- `--animation-fps` : Frames per second in the generated animation (default: 5)

GPU Acceleration

PyTopo3D supports GPU acceleration using NVIDIA CUDA GPUs via the CuPy library. This can significantly speed up the optimization process, especially for large models.

Installation with GPU Support

To use GPU acceleration, install PyTopo3D with GPU support:

```
# Via pip  
pip install pytopo3d[gpu]  
  
# For development installation  
cd build-tools  
  
pip install -e ".[gpu]"
```



This will install the required `cupy` dependency. Make sure to use the appropriate CUDA version that matches your system (e.g., `cupy-cuda11x` for CUDA 11.x, `cupy-cuda12x` for CUDA 12.x).

Enabling GPU Acceleration

By default, GPU acceleration is disabled even if a compatible GPU is available in your system. You can enable it in two ways:

1. Command Line Interface:

```
python main.py --gpu --nelx 64 --nely 32 --nelz 32 [other options]
```



2. Python API:

```
from pytopo3d.core.optimizer import top3d

# Enable GPU acceleration with the use_gpu parameter
result = top3d(
    nelx=64,
    nely=32,
    nelz=32,
    volfrac=0.3,
    penal=3.0,
    rmin=3.0,
    disp_thres=0.5,
    use_gpu=True # Set to True to enable GPU acceleration
)
```



GPU acceleration primarily impacts the linear solver step (the most computationally intensive part) and sensitivity filtering operations.

Experiment Management

PyTopo3D includes a robust experiment management system that automatically:

- Creates a uniquely named directory for each optimization run
- Saves all relevant parameters, inputs, and outputs
- Generates detailed logs of the optimization process
- Records performance metrics and convergence data

Command Line Options

```
python main.py --experiment-name custom_name \  
               --description "Detailed description of this experiment" \  
               --log-level DEBUG \  
               --log-file custom_log.log \  
               --verbose
```



Options:

- --experiment-name : Custom name for the experiment (optional, auto-generated if not provided)
- --description : Description of the experiment stored in the results metadata
- --log-level : Set logging detail level (DEBUG, INFO, WARNING, ERROR, CRITICAL)
- --log-file : Path to a custom log file
- --verbose : Enable more detailed output (sets log level to DEBUG)
- --quiet : Reduce output verbosity (sets log level to WARNING)

Acknowledgements

This code is adapted from [Liu & Tovar's MATLAB code](#) for 3D topology optimization.

K. Liu and A. Tovar, "An efficient 3D topology optimization code written in Matlab", Struct Multidisc Optim, 50(6): 1175-1196, 2014, doi:10.1007/s00158-014-1107-x

Citation

If you use PyTopo3D in your research or work, please cite our paper on ArXiv: [PyTopo3D: A Python Framework for 3D SIMP-based Topology Optimization](#)

Kim, J. & Kang, N. (2024). PyTopo3D: A Python Framework for 3D SIMP-based Topology Optimization. arXiv preprint arXiv:2504.05604.

```
@article{kim2025pytopo3d
  title={PyTopo3D: A Python Framework for 3D SIMP-based Topology Optimization},
  author={Jihoon Kim and Namwoo Kang},
  journal={arXiv preprint arXiv:2504.05604},
  year={2025}
}
```



This paper provides a detailed explanation of the implementation, theoretical foundations, and optimizations used in PyTopo3D. Proper citation helps support the continued development of open-source scientific software.

Roadmap